

Muhammad Iqbal Zafarullah G1A024007

Achmad Apanza G1A024015

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

FAKULTAS TEKNIK

UNIVERSITAS BENGKULU

2026

Akses kode lengkap, dataset, dan seluruh keluaran. Laporan ini dibatasi 8 halaman sehingga hanya memuat tangkapan layar sel yang menjawab langsung poin penilaian. Kode lengkap 17 sel beserta seluruh keluarannya yang telah dijalankan penuh (Runtime > Run all) dapat diperiksa pada notebook Google Colab berikut: <https://colab.research.google.com/drive/12jwFjPK0M2iLoj8Kbu3eg11WibzVMMMO?usp=sharing>.
Berkas mahasiswa_lulus.csv dan notebook .ipynb juga diunggah pada repositori GitHub: https://github.com/MIqbalZafarullah/Tugas_Machine_Learning
Dataset bersifat sintesis sehingga tidak memuat data pribadi mahasiswa sesungguhnya.

A.1 Kapan Decision Tree Lebih Cocok Dibanding Naive Bayes

Alasan pertama: Decision Tree tidak mengasumsikan independensi antar fitur. Naive Bayes bekerja dengan asumsi bahwa semua fitur input bersifat kondisional independen satu sama lain, artinya nilai satu fitur tidak boleh memengaruhi atau berkorelasi dengan fitur lain. Namun dalam dataset mahasiswa ini, asumsi tersebut hampir pasti dilanggar. Misalnya, IPK dan Jam_Belajar sangat mungkin berkorelasi: mahasiswa yang belajar lebih banyak jam cenderung meraih IPK lebih tinggi. Begitu pula Penghasilan_Ortu dan Status_Beasiswa kemungkinan besar berkorelasi negatif karena beasiswa sering diberikan kepada mahasiswa dari keluarga kurang mampu. Ketika asumsi independensi dilanggar seperti ini, Naive Bayes akan menghasilkan estimasi probabilitas yang bias dan tidak akurat. Decision Tree sebaliknya tidak memerlukan asumsi tersebut, ia hanya mencari threshold terbaik pada satu fitur di setiap percabangan, sehingga lebih robust terhadap korelasi antar fitur..

Alasan kedua: Decision Tree menghasilkan model yang mudah diinterpretasi dan dapat dijelaskan secara langsung. Dalam konteks startup EduTech, hasil prediksi perlu dikomunikasikan kepada dosen, konselor akademik, atau bahkan mahasiswa itu sendiri, yang umumnya bukan audiens teknis. Decision Tree menghasilkan aturan keputusan yang sangat intuitif, misalnya: "Jika $IPK \geq 3.0$ DAN Kehadiran $\geq 75\%$, maka mahasiswa diprediksi LULUS TEPAT WAKTU." Aturan semacam ini mudah dipahami, divalidasi, dan ditindaklanjuti sebagai kebijakan. Naive Bayes sebaliknya hanya menghasilkan nilai probabilitas akhir tanpa penjelasan logis tentang variabel mana yang paling menentukan dan bagaimana interaksinya, menjadikannya kurang actionable untuk keperluan intervensi akademik.

A.2 Dua Pasang Fitur yang Melanggar Asumsi Independensi

Pasang pertama, IPK dan Jam_Belajar. Kedua fitur ini hampir pasti berkorelasi positif secara langsung. Mahasiswa yang mendedikasikan lebih banyak waktu untuk belajar secara konsisten cenderung memiliki pemahaman materi yang lebih baik, yang pada akhirnya tercermin dalam nilai dan IPK yang lebih tinggi. Sebaliknya, mahasiswa dengan IPK rendah kemungkinan besar juga memiliki jam belajar yang rendah. Dengan kata lain, jika kita sudah mengetahui bahwa seorang mahasiswa memiliki Jam_Belajar yang tinggi, informasi tersebut sudah memberi kita petunjuk kuat tentang nilai IPK-nya, dan inilah yang dimaksud dengan pelanggaran asumsi independensi. Naive Bayes akan menghitung kontribusi kedua fitur ini secara terpisah seolah-olah tidak ada hubungan, sehingga secara tidak langsung menghitung dua kali bobot dari informasi yang sama dan mendistorsi nilai probabilitas akhir.

Pasang kedua, Penghasilan_Ortu dan Status_Beasiswa. Di Indonesia, beasiswa, khususnya beasiswa kebutuhan (need-based scholarship) seperti KIP-Kuliah, secara eksplisit mensyaratkan bahwa penerima berasal dari keluarga dengan penghasilan rendah. Ini berarti ada korelasi negatif yang sangat kuat antara Penghasilan_Ortu dan kemungkinan mendapat beasiswa: semakin rendah penghasilan orang tua, semakin tinggi peluang mahasiswa memiliki beasiswa. Kedua fitur ini jelas tidak independen karena satu fitur bisa diprediksi dengan cukup baik hanya dari fitur lainnya. Jika Naive Bayes memperlakukan keduanya sebagai kontributor independen terhadap prediksi kelulusan, model akan salah mengukur pengaruh kondisi finansial mahasiswa karena efek finansial dihitung dua kali melalui dua fitur yang sebenarnya mengukur hal yang sama dari sudut berbeda.

A.3 Risiko Overfitting pada Decision Tree dan Cara Mengatasinya

Risiko. Overfitting adalah kondisi di mana model terlalu "menghafal" data training hingga kehilangan kemampuan generalisasi ke data baru. Pada dataset ini dengan hanya 500 sampel dan 7 fitur, Decision Tree memiliki risiko overfitting yang cukup nyata. Tanpa pembatasan apapun, algoritma Decision Tree akan

terus melakukan pembelahan (splitting) hingga setiap leaf node hanya berisi satu atau beberapa sampel yang homogen. Hasilnya adalah pohon yang sangat dalam dan kompleks dengan ratusan cabang yang mencerminkan kekhasan, bahkan noise, dalam data training. Model seperti ini bisa mencapai akurasi training mendekati 100%, namun ketika diuji pada data baru yang belum pernah dilihat, performanya anjlok drastis. Pada dataset kecil seperti 500 mahasiswa, risiko ini semakin besar karena variasi statistik antar sampel relatif tinggi; pola yang tampak "nyata" dalam training bisa jadi hanyalah kebetulan statistik yang tidak akan muncul lagi di data lain.

Cara mengatasi. Pertama, pre-pruning atau pembatasan pertumbuhan pohon sejak awal dengan mengatur hyperparameter seperti `max_depth` (kedalaman maksimum pohon), `min_samples_split` (jumlah minimum sampel agar sebuah node boleh dipecah), dan `min_samples_leaf` (jumlah minimum sampel yang harus ada di setiap leaf). Misalnya, menetapkan `max_depth=5` dan `min_samples_leaf=10` akan mencegah pohon tumbuh terlalu spesifik. Kedua, post-pruning yaitu membiarkan pohon tumbuh penuh terlebih dahulu, lalu memangkas cabang-cabang yang tidak memberikan peningkatan signifikan pada validasi set, teknik ini dikenal sebagai cost-complexity pruning. Ketiga, penggunaan k-fold cross-validation untuk memilih kombinasi hyperparameter yang optimal secara objektif, bukan hanya berdasarkan performa di training set.

BAGIAN B. IMPLEMENTASI DAN EVALUASI – TUGAS KELOMPOK

Implementasi dikerjakan pada Google Colab dengan dataset dibaca langsung dari Google Drive, `random_state` dikunci pada 42 agar seluruh hasil dapat direproduksi. Notebook telah dijalankan penuh melalui Runtime > Run all. Tangkapan layar sel persiapan lingkungan, eksplorasi data, encoding, penyimpanan luaran, dan pengujian pembandingan Pipeline tidak disertakan di sini karena keterbatasan halaman, dan seluruhnya dapat diperiksa pada notebook Colab yang tautannya tercantum pada halaman pertama laporan ini.

B.1 Preprocessing

Pembagian data dilakukan lebih dahulu, kemudian nilai hilang diisi menggunakan statistik yang dihitung hanya dari data latih. Apabila median dan modus dihitung dari keseluruhan 500 baris, informasi data uji akan ikut memengaruhi pelatihan dan menimbulkan kebocoran data. Data terbagi menjadi 400 data latih dan 100 data uji sesuai ketentuan 80:20, dengan `stratify=y` sehingga proporsi kelas positif terjaga pada 0,705 di data latih dan 0,700 di data uji. Nilai pengisi dari data latih adalah median IPK 2,92, Kehadiran 72,0, Jam_Belajar 11,8, serta modus Organisasi Ya, Penghasilan_Ortu Menengah, Jenis_Kelamin P, dan Status_Beasiswa Tidak. Median dipilih karena lebih tahan pencilan, sedangkan modus dipakai untuk data nominal yang tidak memiliki nilai tengah. Setelah imputasi, jumlah sel kosong pada kedua himpunan bernilai nol. Encoding dibedakan menurut sifat fitur: Organisasi, Status_Beasiswa, dan Jenis_Kelamin dipetakan biner menjadi 0 dan 1, sedangkan Penghasilan_Ortu dipetakan ordinal menjadi Rendah 0, Menengah 1, dan Tinggi 2 karena kategorinya berjenjang.

```
X_raw = df[num_cols + cat_cols].copy()
y = (df[TARGET] == "Ya").astype(int) # 1 = lulus tepat waktu

# (c) split 80:20 - DILAKUKAN SEBELUM imputasi, agar tidak data leakage
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X_raw, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y)

print("Train:", X_train_raw.shape[0], " | Test:", X_test_raw.shape[0])
print("Proporsi kelas 1 -> train %.3f | test %.3f" % (y_train.mean(), y_test.mean()))

# (a) missing value: median & modus DARI DATA LATIH SAJA
impute_num = (c: X_train_raw[c].median() for c in num_cols)
impute_cat = (c: X_train_raw[c].mode()[0] for c in cat_cols)
print("\nNilai pengisi:", impute_num, impute_cat)

X_train_imp = X_train_raw.fillna(**impute_num, **impute_cat)
X_test_imp = X_test_raw.fillna(**impute_num, **impute_cat)
print("Sisa NaN -> train:", int(X_train_imp.isna().sum().sum()), " | test:", int(X_test_imp.isna().sum().sum()))

Train: 400 | Test: 100
Proporsi kelas 1 -> train 0.705 | test 0.700

Nilai pengisi: {'IPK': 2.92, 'Kehadiran': 72.0, 'Jam_Belajar': 11.8} {'Organisasi': 'Ya', 'Penghasilan_Ortu': 'Menengah', 'Jenis_Kelamin': 'P', 'Status_Beasiswa': 'Tidak'}
Sisa NaN -> train: 0 | test: 0
```

```

# (b) encoding: biner + ordinal
MAPS = {
    "Organisasi": {"Tidak": 0, "Ya": 1},
    "Status_Beasiswa": {"Tidak": 0, "Ya": 1},
    "Jenis_Kelamin": {"L": 0, "P": 1},
    "Penghasilan_Ortu": {"Rendah": 0, "Menengah": 1, "Tinggi": 2}, # ordinal, berjenjang
}

def encode(dfx):
    out = dfx.copy()
    for col, m in MAPS.items():
        out[col] = out[col].map(m).astype(int)
    return out

X_train, X_test = encode(X_train_imp), encode(X_test_imp)
FEATURES = list(X_train.columns)

print("Fitur:", FEATURES)
display(X_train.head())
assert X_train.isna().sum().sum() == 0
print("Preprocessing selesai.")

--- Fitur: ['IPK', 'Kehadiran', 'Jam_Belajar', 'Organisasi', 'Penghasilan_Ortu', 'Jenis_Kelamin', 'Status_Beasiswa']
IPK  Kehadiran  Jam_Belajar  Organisasi  Penghasilan_Ortu  Jenis_Kelamin  Status_Beasiswa
209  3.00      58.0        9.0          1          1          1          0
487  3.03      88.0       11.4          1          0          1          0
168  3.57      57.0       11.6          1          2          0          0
238  2.92      48.0        9.3          0          1          0          0
451  2.13      62.0       14.4          0          0          1          0
Preprocessing selesai.

```

B.2 Modeling dan Bukti Overfitting

Tiga model dibangun sesuai ketentuan, yaitu DecisionTreeClassifier dengan max_depth=3, DecisionTreeClassifier dengan max_depth=None, dan GaussianNB. Perbedaan ukuran pohon sangat tajam: max_depth=3 hanya menghasilkan 8 daun, sedangkan tanpa batas kedalaman pohon tumbuh hingga kedalaman 18 dengan 90 daun. Rasio 90 daun terhadap 400 data latih berarti setiap daun rata-rata hanya menampung 4 sampel, sehingga model menghafal alih-alih menggeneralisasi. Hal ini terlihat dari akurasi latih 1,0000 yang justru turun menjadi 0,6800 pada data uji sehingga menghasilkan selisih 0,3200, sementara GaussianNB memiliki selisih nol.

```

dt3 = DecisionTreeClassifier(max_depth=3, random_state=RANDOM_STATE)
dt_full = DecisionTreeClassifier(max_depth=None, random_state=RANDOM_STATE)
gnb = GaussianNB()

models = {
    "Decision Tree (max_depth=3)": dt3,
    "Decision Tree (max_depth=None)": dt_full,
    "Gaussian Naive Bayes": gnb,
}

for nama, m in models.items():
    m.fit(X_train, y_train); print("[OK]", nama)

print("\nUkuran pohon:")
print("  depth=3  -> kedalaman 3, daun 8 X (dt3.get_depth(), dt3.get_n_leaves())")
print("  depth=None -> kedalaman 18, daun 90 X (dt_full.get_depth(), dt_full.get_n_leaves())")

# bukti overfitting
gap = pd.DataFrame([
    "Model": m,
    "Train Acc": round(accuracy_score(y_train, m.predict(X_train)), 4),
    "Test Acc": round(accuracy_score(y_test, m.predict(X_test)), 4),
    for m, m in models.items()])
gap["Gap"] = (gap["Train Acc"] - gap["Test Acc"]).round(4)
display(gap)

--- [OK] Decision Tree (max_depth=3)
[OK] Decision Tree (max_depth=None)
[OK] Gaussian Naive Bayes

Ukuran pohon:
depth=3  -> kedalaman 3, daun 8
depth=None -> kedalaman 18, daun 90

Model  Train Acc  Test Acc  Gap
0  Decision Tree (max_depth=3)  0.7875  0.71  0.0775
1  Decision Tree (max_depth=None)  1.0000  0.68  0.3200
2  Gaussian Naive Bayes  0.7800  0.78  0.0000

```

B.3 Evaluasi

Model	Akurasi	Precision	Recall	F1	F1-Macro	Gap	TN/FP/FN/TP	CV F1-Macro
DT depth=3	0,7100	0,7470	0,8857	0,8105	0,5967	0,0775	9/21/8/62	0,6058 +/- 0,0252
DT depth=None	0,6800	0,7794	0,7571	0,7681	0,6260	0,3200	15/15/17/53	0,6344 +/- 0,0346
GaussianNB	0,7500	0,8358	0,8000	0,8175	0,7103	0,0000	19/11/14/56	0,7015 +/- 0,0129

Tabel 2. Metrik evaluasi, selisih latih-uji, confusion matrix, dan validasi silang lima lipatan

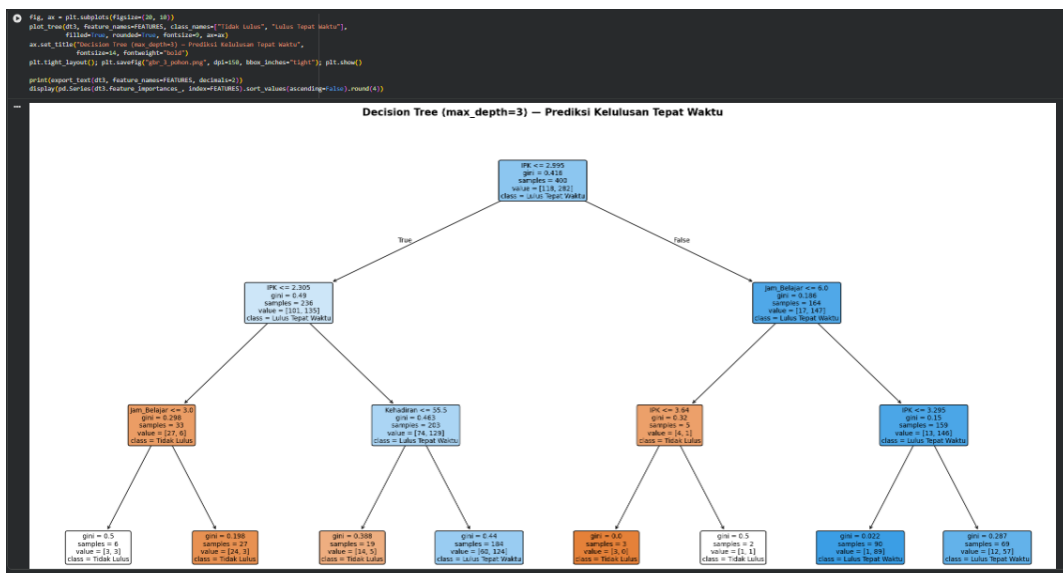
GaussianNB unggul pada empat dari lima metrik. Pohon max_depth=3 memang mencatat recall tertinggi 0,8857, tetapi angka itu menyesatkan bila dibaca terpisah: pada kelas Tidak Lulus Tepat Waktu recall-nya hanya 0,30, artinya sekitar 70 persen mahasiswa berisiko tidak terdeteksi, sedangkan GaussianNB mencapai 0,63 pada kelas yang sama. Confusion matrix memperkuat hal ini, karena pohon max_depth=3 menghasilkan 21 False Positive sementara GaussianNB hanya 11. Validasi silang lima

lipatan mengonfirmasi keunggulan tersebut secara lebih stabil dengan F1-Macro 0,7015 dan simpangan baku 0,0129, yang terkecil di antara ketiga model, sehingga keunggulannya bukan kebetulan satu pembagian data.



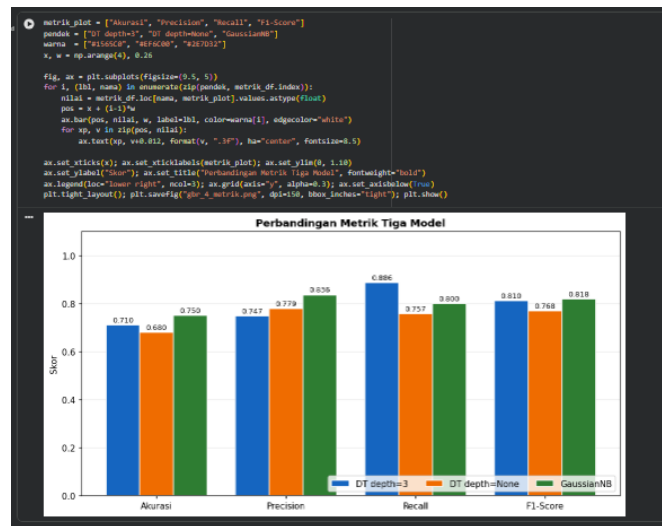
B.4 Visualisasi

Akar pohon membelah pada kondisi IPK kurang dari atau sama dengan 2,995 terhadap 400 sampel dengan gini 0,416. Cabang kiri memisah pada IPK 2,305 lalu Kehadiran 55,5, sedangkan cabang kanan memisah pada Jam_Belajar 6,0 lalu IPK 3,295. Nilai feature importance menunjukkan IPK mendominasi dengan 0,7384, diikuti Jam_Belajar 0,1380 dan Kehadiran 0,1236, sedangkan empat fitur kategorik seluruhnya bernilai 0,0000. Artinya pohon sama sekali tidak menggunakan Penghasilan_Ortu maupun Jenis_Kelamin, temuan yang menjadi dasar pembahasan etika pada Bagian C. Diagram batang menegaskan GaussianNB unggul pada akurasi, precision, dan F1-Score, sementara pohon tanpa batas kedalaman terendah pada tiga dari empat metrik.



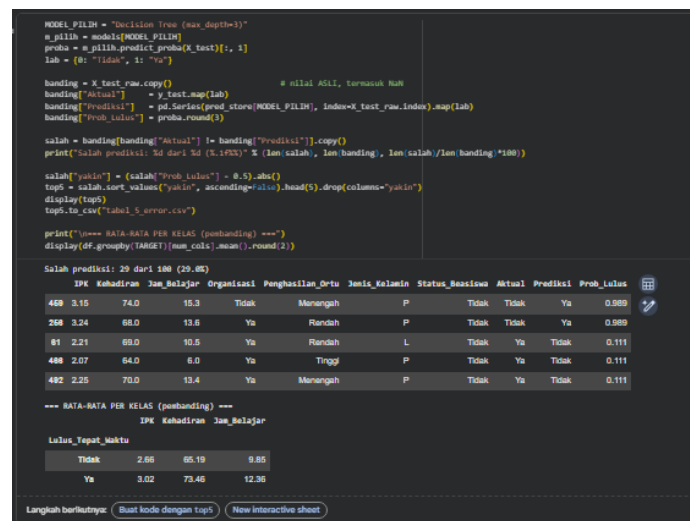
Pohon pada Gambar dapat dibaca sebagai aturan berjenjang yang mudah dikomunikasikan kepada pihak kampus. Mahasiswa dengan IPK di atas 2,995 dan Jam_Belajar di atas 6 jam per minggu langsung

jatuh ke daun berlabel Ya dengan kemurnian sangat tinggi, yaitu daun bernilai [1, 89] dengan gini 0,022. Sebaliknya, cabang berisiko muncul ketika IPK berada di bawah 2,305 atau ketika Kehadiran berada di bawah 55,5 persen, dan daun penyumbang kesalahan terbesar bernilai [12, 57] yang menjelaskan tingginya False Positive pada model ini. Kedalaman tiga sudah cukup memisahkan mayoritas kasus sehingga penambahan kedalaman hanya menambah daun kecil tanpa memperbaiki generalisasi.



B.5 Analisis Error

Dari 100 data uji, Decision Tree max_depth=3 salah memprediksi 29 data atau 29,0 persen. Lima kasus berikut dipilih berdasarkan keyakinan model tertinggi. Sebagai acuan, rata-rata kelas Tidak adalah IPK 2,66, Kehadiran 65,19, dan Jam_Belajar 9,85, sedangkan kelas Ya adalah 3,02, 73,46, dan 12,36.



Indeks 468 dan 268 (False Positive, P = 0,989). Keduanya memiliki IPK 3,15 dan 3,24 dengan kehadiran serta jam belajar di atas rata-rata kelas Ya, sehingga jatuh pada daun berkomposisi 12 berbanding 57 yang didominasi kelas lulus. Penyebab ketidakkelulusannya tidak terekam sebagai fitur, misalnya kondisi kesehatan atau kendala tugas akhir. Pada indeks 268, Penghasilan_Ortu Rendah tidak pernah dipakai pohon karena importance-nya nol, padahal tekanan ekonomi berpotensi memaksa mahasiswa bekerja sambilan.

Indeks 81, 408, dan 492 (False Negative, P = 0,111). Ketiganya memiliki IPK di bawah 2,31 sehingga masuk cabang IPK kurang dari 2,305 yang seluruh daunnya berkelas Tidak. Kehadiran 69,0 pada indeks 81 dan Jam_Belajar 13,4 pada indeks 492 yang sebenarnya di atas rata-rata kelas Ya tidak lagi diperhitungkan, karena pohon sudah memutuskan arah prediksi pada tingkat pertama. Inilah kelemahan struktur hierarkis yang serakah, yaitu keputusan di tingkat atas tidak dapat dianulir bukti di tingkat bawah.

Pola umum. Dua False Positive terjadi pada mahasiswa dengan seluruh indikator akademik tinggi, sedangkan ketiga False Negative seluruhnya berIPK di bawah 2,31. Pola ini menegaskan dominasi IPK sebesar 0,7384: ketika IPK bertentangan dengan fitur lain, pohon dangkal hampir selalu memihak IPK. Implikasinya, prediksi model layak diperlakukan sebagai penyaring awal, bukan keputusan final.

BAGIAN C. REKOMENDASI DAN ETIKA – TUGAS KELOMPOK

C.1 Rekomendasi Model

Model yang direkomendasikan adalah **Gaussian Naive Bayes**, berdasarkan lima bukti kuantitatif pada Tabel 2. Pertama, GaussianNB unggul pada empat dari lima metrik dengan akurasi 0,7500, precision 0,8358, F1-Score 0,8175, dan F1-Macro 0,7103. Kedua, selisih akurasi latih dan uji bernilai 0,0000 sehingga tidak mengalami overfitting, berbeda jauh dari pohon tanpa batas kedalaman yang selisihnya 0,3200. Ketiga, validasi silang lima lipatan memberikan F1-Macro 0,7015 dengan simpangan baku 0,0129 yang merupakan terkecil, sehingga performanya tidak bergantung pada satu pembagian data yang menguntungkan. Keempat, recall kelas minoritas mencapai 0,63 dibanding hanya 0,30 pada pohon `max_depth=3`, padahal kelas inilah tujuan utama sistem karena kampus membangun model untuk menemukan mahasiswa berisiko. Kelima, False Positive hanya 11 dibanding 21, dan setiap False Positive berarti satu mahasiswa berisiko yang lolos dari pendampingan.

Pohon tanpa batas kedalaman ditolak karena terbukti menghafal data latih melalui 90 daun pada kedalaman 18. Pohon `max_depth=3` tidak dipilih sebagai model utama karena nyaris buta terhadap kelas minoritas, namun tetap disarankan dipertahankan sebagai **model pendamping** untuk keperluan penjelasan, sebab aturannya yang hanya 8 daun dapat dibaca langsung oleh dosen pembimbing tanpa pengetahuan statistik. Skema yang diusulkan adalah GaussianNB sebagai mesin penilai risiko dan pohon `max_depth=3` sebagai alat komunikasi keputusan.

C.2 Bias dan Etika pada Fitur Penghasilan_Ortu dan Jenis_Kelamin

Feature importance kedua pohon menunjukkan Penghasilan_Ortu dan Jenis_Kelamin bernilai 0,0000, sehingga Decision Tree secara struktural bebas dari bias kedua fitur tersebut. Namun GaussianNB tidak memiliki mekanisme seleksi fitur karena seluruh fitur ikut dikalikan ke dalam probabilitas posterior, sehingga pada model yang justru direkomendasikan kedua fitur itu tetap memengaruhi keputusan. Risiko bias karenanya nyata, bukan sekadar kemungkinan teoretis.

1. **Diskriminasi melalui variabel pengganti.** Tingkat penerimaan beasiswa tercatat 37,4 persen pada penghasilan rendah, 31,1 persen menengah, dan 17,3 persen tinggi, dengan Cramer's V 0,157. Model dapat mempelajari pola bahwa mahasiswa dari keluarga berpenghasilan rendah lebih berisiko, sehingga keputusan yang seharusnya berbasis kemampuan akademik berubah menjadi keputusan berbasis kelas ekonomi.
2. **Lingkaran umpan balik yang merugikan.** Jika mahasiswa berpenghasilan rendah diprediksi tidak lulus tepat waktu lalu beasiswanya dicabut, tekanan ekonominya meningkat dan peluang kelulusannya benar-benar menurun. Data tersebut lalu dipakai melatih ulang model dan memperkuat keyakinan awal, sehingga terjadi ramalan yang memenuhi dirinya sendiri.
3. **Pembalikan tujuan beasiswa.** Beasiswa adalah instrumen pemerataan kesempatan, tetapi bila model dipakai sebagai penyaring, kelompok yang paling membutuhkan justru paling sering tersaring keluar. Rata-rata IPK penerima beasiswa 3,088 dibanding 2,838 pada bukan penerima menunjukkan beasiswa terbukti membantu capaian akademik.
4. **Tingkat kesalahan yang belum layak.** Akurasi model terbaik hanya 0,7500 atau salah pada sekitar satu dari empat keputusan. Bagian B membuktikan ada mahasiswa berIPK 2,07 yang tetap lulus tepat waktu dan mahasiswa berIPK 3,24 yang justru tidak, sehingga keputusan yang memengaruhi pembiayaan pendidikan tidak dapat dipertanggungjawabkan.

Strategi Mitigasi	Bentuk Pelaksanaan	Dasar Bukti
Hapus fitur sensitif	Latih ulang tanpa Penghasilan_Ortu dan Jenis_Kelamin	Importance keduanya 0,0000, performa pohon tidak berubah
Audit keadilan	Hitung recall dan false positive rate per subkelompok	Metrik agregat 0,7500 menyembunyikan selisih antar-subkelompok

Manusia penentu akhir	Prediksi hanya menjadi daftar prioritas tinjauan dosen	Terdapat 29 kesalahan dari 100 data uji
Batasi kegunaan model	Hanya untuk intervensi positif, dilarang mencabut beasiswa	21 dan 11 False Positive berisiko salah cabut
Transparansi berkala	Dokumentasi model, hak sanggah mahasiswa, audit per semester	Aturan pohon 8 daun mudah dikomunikasikan

Tabel 3. Strategi mitigasi bias beserta dasar buktinya

Mitigasi utama adalah menghapus kedua fitur sensitif dari pelatihan. Langkah ini biasanya menimbulkan pertukaran antara keadilan dan akurasi, namun di sini biayanya nyaris nol karena importancenya sudah 0,0000 pada pohon. Untuk GaussianNB, penghapusan perlu diikuti pengujian ulang guna memastikan penurunan F1-Macro dari 0,7103 masih dapat diterima. Penghapusan fitur saja tidak cukup, sebab IPK dan Kehadiran sendiri masih berpotensi menjadi variabel pengganti bagi kondisi ekonomi, sehingga audit keadilan per subkelompok tetap wajib dijalankan.

C.3 Simulasi Mahasiswa IPK Tinggi dengan Kehadiran Rendah

Profil yang diuji adalah mahasiswa dengan IPK 3,75 yang sangat tinggi namun Kehadiran hanya 55 persen, dengan Jam_Belajar 8 jam per minggu. Hasil eksekusi disajikan pada Tabel 4 beserta uji sensitivitasnya terhadap perubahan nilai kehadiran.

Model	Prediksi	Keh. 45%	Keh. 55%	Keh. 65%	Keh. 75%	Keh. 85%
DT depth=3	Ya (0,826)	0,826	0,826	0,826	0,826	0,826
DT depth=None	Tidak (0,000)	-	0,000	-	-	-
GaussianNB	Ya (0,719)	0,437	0,719	0,826	0,934	0,960

Tabel 4. Prediksi ketiga model pada kasus simulasi dan sensitivitasnya terhadap Kehadiran (IPK 3,75)

Ketiga model memberi hasil berbeda karena cara kerjanya berbeda secara mendasar. Decision Tree membelah ruang fitur secara hierarkis dan serakah, sehingga sebuah fitur hanya diperiksa apabila jalur pengujian melewati cabang yang memuatnya. Pada pohon max_depth=3, Kehadiran hanya muncul sekali dengan ambang 55,5 dan letaknya di cabang IPK rendah. Karena mahasiswa simulasi berIPK 3,75, penelusuran mengambil arah kanan pada akar, melewati cabang Jam_Belajar di atas 6,0, lalu berakhir pada daun berdasarkan IPK 3,295, sehingga Kehadiran tidak pernah diperiksa. Sebaliknya Naive Bayes mengalikan likelihood seluruh fitur secara serentak, dan Kehadiran 55 persen yang jauh di bawah rata-rata kelas lulus sebesar 73,46 menarik turun probabilitas akhir menjadi 0,719.

Tabel 4 memberi bukti paling tegas. Ketika Kehadiran diubah dari 45 hingga 85 persen, probabilitas pohon max_depth=3 sama sekali tidak bergerak dari 0,826, sedangkan GaussianNB naik lebih dari dua kali lipat dari 0,437 menjadi 0,960. Bagi mahasiswa berIPK tinggi, pohon dangkal memperlakukan kehadiran 45 persen dan 85 persen sebagai kasus identik, dan hal ini berbahaya secara kebijakan karena kehadiran 45 persen justru berisiko terkena sanksi akademik. Adapun probabilitas 0,000 pada pohon tanpa batas kedalaman bukan tanda keyakinan yang beralasan, melainkan akibat kombinasi fitur simulasi jatuh pada daun murni yang dibentuk hanya oleh satu atau dua sampel, sejalan dengan selisih latih-uji 0,3200. Implikasi praktisnya, untuk profil dengan indikator saling bertentangan GaussianNB lebih dapat dipercaya, prediksi Ya tidak boleh dibaca sebagai jaminan aman, dan kampus sebaiknya menerapkan aturan tambahan di luar model seperti menandai setiap mahasiswa dengan kehadiran di bawah 60 persen tanpa memandang IPK.

KESIMPULAN

Gaussian Naive Bayes dipilih sebagai model utama dengan akurasi 0,7500, F1-Macro 0,7103, validasi silang 0,7015 plus minus 0,0129, dan tanpa gejala overfitting. Pohon keputusan max_depth=3 dipertahankan sebagai alat penjelas karena aturannya hanya 8 daun, sementara pohon tanpa batas kedalaman ditolak karena selisih latih-uji 0,3200. Fitur Penghasilan_Ortu dan Jenis_Kelamin direkomendasikan dihapus dari pelatihan karena kontribusinya nol pada pohon namun tetap berisiko menimbulkan bias pada Naive Bayes. Dengan tingkat kesalahan 29 dari 100 data uji, model ini hanya layak digunakan sebagai sistem peringatan dini untuk menyalurkan pendampingan akademik, dan tidak layak dijadikan dasar keputusan pencabutan beasiswa.